

Project Documentation

College Support Multi-Agent Portal

A complete guide explaining how the project works, how its modules connect, and how each user workflow moves through the system.

Frontend

Next.js and React TypeScript application in `web/`.

Backend

NestJS TypeScript API in `api/`.

Database

PostgreSQL with Prisma ORM and pgvector support.

AI Design

Multi-agent assistant with orchestrator, knowledge, and workflow agents.

Generated from the current repository structure. Main source files include `README.md`, `api/prisma/schema.prisma`, `api/src`, and `web/src`.

Table Of Contents

- | | |
|-----------------------------------|---------------------------------------|
| 1. Project Purpose | 9. Knowledge Base And RAG Flow |
| 2. High-Level Architecture | 10. Ticket Workflow |
| 3. Technology Stack | 11. Admin Workflow |
| 4. User Roles | 12. Security And Permissions |
| 5. Database Design | 13. Notifications, Email, And Uploads |
| 6. Backend API Modules | 14. Public API |
| 7. Frontend Application | 15. Deployment And Running |
| 8. Assistant And Multi-Agent Flow | 16. Testing And Project Health |
| | 17. Recommended Improvements |

1. Project Purpose

The College Support Multi-Agent Portal is a support system for university students, staff, and admins. Its main goal is to reduce manual reception work by letting students ask common questions, receive answers from trusted knowledge documents, and create tickets when the assistant cannot safely answer.

The project focuses on two support areas: **REGISTRATION** and **IT**. It combines a normal support-ticket workflow with an AI assistant. The assistant does not answer freely from memory only. It searches the local knowledge base first, calculates confidence, and escalates uncertain cases into tickets.

Main idea

Students get fast self-service answers when trusted information exists. Staff get structured tickets when human review is required. Admins manage users, knowledge content, and operational visibility.

2. High-Level Architecture

The application is split into four major layers: browser frontend, backend API, database/storage, and optional external services.

Browser

- > Next.js web app
- > NestJS REST API
- > Prisma ORM
- > PostgreSQL database

Assistant request

```
-> AssistantController
-> AssistantService
-> OrchestratorService
-> KnowledgeAgentService and/or WorkflowAgentService
-> KnowledgeService, GeminiService, TicketsService
-> Database records and response back to the user
```

Layer Responsibilities

Layer	Responsibility
Web app	Displays login, registration, dashboard, assistant chat, ticket lists, ticket details, admin panels, and forms.
API	Handles authentication, authorization, assistant routing, tickets, files, notifications, knowledge management, and public endpoints.
Database	Stores users, profiles, tickets, events, attachments, knowledge documents, FAQ entries, AI traces, API keys, and notifications.
External services	Gemini can provide embeddings and answers. SMTP can send ticket email notifications. Both have local/fallback behavior where possible.

3. Technology Stack

Frontend

- Next.js 16
- React 19
- TypeScript
- Local browser storage for JWT token

Backend

- NestJS 11
- Prisma ORM
- PostgreSQL 16
- JWT authentication and role guards

AI And Search

- Gemini text generation
- Gemini embeddings
- Local deterministic fallback when `GEMINI_API_KEY` is missing
- Keyword overlap and cosine similarity ranking

Deployment

- Docker Compose
- Nginx reverse proxy
- Persistent database volume
- Persistent upload volume

4. User Roles

The application uses three roles: `STUDENT`, `STAFF`, and `ADMIN`. Permissions are enforced in the backend by JWT authentication, role guards, and service-level access checks.

Role	What The Role Can Do	Important Limits
<code>STUDENT</code>	Register, log in, ask the assistant, create tickets, view their own tickets, upload attachments, and send ticket messages.	Students can only access their own tickets. They need an academic department assigned before creating tickets.
<code>STAFF</code>	View queue tickets in their assigned support area and academic department, claim tickets, update status, reply, and manage attachments.	Staff cannot create tickets for themselves. They can only work in their assigned lane.
<code>ADMIN</code>	Provision users, assign routing, manage knowledge content, view traces, and oversee all ticket queues.	Admins have global visibility, so the account must be protected carefully.

5. Database Design

The Prisma schema in `api/prisma/schema.prisma` defines the main business data. The design is relational and keeps audit/history data separate from the main ticket records.

Main Models

Model	Purpose
<code>User</code>	Stores login identity, school ID, email, hashed password, role, support area, academic department, and relations to tickets/events.
<code>Profile</code>	Stores user display information such as full name and avatar URL.
<code>Ticket</code>	Stores support request subject, description, area, department, status, student, assignee, and timestamps.
<code>TicketEvent</code>	Stores ticket history such as creation, claim, status change, update, attachment events, and messages.
<code>Attachment</code>	Stores uploaded file metadata and path to disk storage.
<code>KnowledgeDocument</code>	Stores uploaded or seeded knowledge documents used by the assistant.
<code>KnowledgeChunk</code>	Stores document chunks and embeddings for retrieval.
<code>FaqEntry</code>	Stores admin-created question and answer pairs.

<code>OrchestratorTrace</code>	Stores AI routing decisions, confidence values, routed agents, and outcomes.
<code>ApiKey</code>	Stores hashed API keys for the external public API. The dashboard no longer exposes key generation, but the backend model remains for public API protection.
<code>Notification</code>	Stores unread/read notification records for users.

Important Enums

Role: STUDENT, STAFF, ADMIN

SupportArea: REGISTRATION, IT

AcademicDepartment: ENGINEERING, BUSINESS, LAW, MEDICINE

TicketStatus: OPEN, IN_PROGRESS, RESOLVED

6. Backend API Modules

The backend is organized by domain modules. `api/src/app.module.ts` imports all modules and registers global throttling.

Module	Main Files	What It Does
Auth	<code>auth.service.ts</code> , <code>auth.controller.ts</code>	Student registration, login, admin user provisioning, password hashing with Argon2, and JWT signing.
Users	<code>users.service.ts</code> , <code>users.controller.ts</code>	Returns current user profile and supports profile updates.
Tickets	<code>tickets.service.ts</code> , <code>tickets.controller.ts</code>	Creates tickets, lists student tickets and staff queues, enforces access, handles claims, status updates, messages, and attachments.
Assistant	<code>assistant.service.ts</code> , <code>orchestrator.service.ts</code> , <code>knowledge-agent.service.ts</code> , <code>workflow-agent.service.ts</code>	Classifies messages, routes agents, answers from trusted knowledge, escalates to tickets, and records traces.
Knowledge	<code>knowledge.service.ts</code> , <code>knowledge.controller.ts</code>	Creates FAQ entries, uploads documents, chunks text, stores embeddings, searches, and retrieves relevant context.
AI	<code>gemini.service.ts</code>	Connects to Gemini for embeddings and answer generation, with deterministic fallback when no API key exists.

Admin	<code>admin.controller.ts</code>	Lists users, updates routing assignments, exposes traces, and still contains backend API key creation.
Notifications	<code>notifications.service.ts</code>	Creates notifications, returns unread counts, and marks related ticket notifications as read.
Public API	<code>public-api.controller.ts</code> , <code>api-key.guard.ts</code>	Provides API-key-protected endpoints for external systems.

7. Frontend Application

The frontend lives in `web/src`. It uses Next.js app routes and client components for interactive dashboard workflows.

Main Pages

Page	Purpose
<code>/</code>	Landing page that introduces the support portal.
<code>/login</code>	Authenticates existing users with school ID and password.
<code>/register</code>	Lets students create accounts with school ID, full name, password, and academic department.
<code>/dashboard</code>	Main role-based dashboard for assistant, tickets, queues, admin knowledge tools, user provisioning, and traces.
<code>/dashboard/tickets/[id]</code>	Ticket detail page with messages, attachments, status changes, and ticket event history.
<code>/privacy</code> , <code>/terms</code>	Static legal/support pages.

Frontend Data Flow

```
User logs in
-> login page receives accessToken
-> token stored in localStorage
-> apiRequest adds Authorization: Bearer token
-> dashboard loads /me and notifications
-> role controls which panels are shown
-> actions call NestJS API endpoints
```

The helper in `web/src/lib/api.ts` centralizes API requests, JSON headers, auth token headers, form-data uploads, blob downloads, and error parsing.

8. Assistant And Multi-Agent Flow

The assistant is the core feature. It receives a message, decides whether it is a knowledge question, workflow request, or mixed request, then delegates to the correct agent.

Agents

Agent	Role In The System
-------	--------------------

Orchestrator Agent	Classifies intent using keyword signals and detects support area from the message.
Knowledge Agent	Retrieves trusted context and asks Gemini/local fallback to answer only from that context.
Workflow Agent	Creates tickets or checks ticket status when the user asks for workflow actions.

Assistant Flow

```

POST /api/assistant/message
-> JwtAuthGuard checks the user
-> OrchestratorService.classifyIntent()
-> If KNOWLEDGE or MIXED:
    KnowledgeAgentService.answer()
    KnowledgeService.retrieveRelevant()
    GeminiService.answerWithContext()
    confidence is calculated
-> If low confidence:
    ask student whether to create ticket
    or create ticket immediately when requested
-> If WORKFLOW:
    WorkflowAgentService.handle()
    TicketsService creates/checks ticket
-> OrchestratorTrace is stored
-> response returns message, intent, confidence, citations, and optional ticketId

```

The confidence threshold is `0.35`. Below that, the assistant avoids risky answers and asks to escalate or creates a ticket if the user has requested escalation.

9. Knowledge Base And RAG Flow

The knowledge base is seeded from Markdown files in `docs/knowledge` and can also be expanded by admins through FAQ creation and document upload.

Current Knowledge Sources

- `docs/knowledge/liu-registration-quick-guide.md`
- `docs/knowledge/liu-admission-registration-student-support.md`
- Admin-created FAQ entries stored in `faq_entries`
- Admin-uploaded documents stored in `knowledge_documents` and `knowledge_chunks`

Retrieval Logic

1. The question is embedded using Gemini or local fallback.
2. Document chunks are loaded from active knowledge documents.
3. FAQ entries are converted into `Q: ... A: ...` candidates.

4. Question-answer pairs are extracted from document content.
5. Candidates are scored with cosine similarity, keyword overlap, and small boosts for exact QA-style content.
6. Low-score results are filtered out, duplicates are removed, and the top results are sent to the answer generator.

Safety behavior

If no trusted context is found, or the model returns `CONTEXT_INSUFFICIENT`, the assistant does not invent an answer. It asks to create a ticket or escalates directly when requested.

10. Ticket Workflow

Tickets are the human escalation path. They can be created directly by students or indirectly by the assistant when trusted knowledge is insufficient.

Student Ticket Creation

```
Student submits ticket
-> POST /api/tickets
-> TicketsService checks:
    user exists
    role is STUDENT
    account is active
    academicDepartment is assigned
-> Ticket is created with status OPEN
-> TicketEvent CREATED is stored
-> matching staff receive notifications and email
-> ticket appears in student dashboard and staff queue
```

Staff Queue And Claiming

Staff members only see tickets in their assigned `supportArea` and `academicDepartment`. A ticket can be unassigned or assigned to the current staff member. If another staff member already claimed it, access is blocked.

Status And Messages

- `OPEN` means the ticket is waiting or unclaimed.
- `IN_PROGRESS` means staff has claimed or started working on it.
- `RESOLVED` means the support request is complete.
- Messages are stored as `TicketEvent` records with event type `MESSAGE`.
- Every meaningful action creates a ticket event so the history is visible.

Attachments

Attachments are uploaded through multipart form data. The backend accepts PNG, JPEG, PDF, and text files up to 5 MB. Metadata is stored in the database and the physical file is stored in the upload directory.

11. Admin Workflow

Admins manage the operational side of the portal. Their dashboard includes knowledge tools, user provisioning, user routing, and trace review.

Admin Feature	How It Works
---------------	--------------

Provision users	Admin creates staff or admin accounts through <code>/auth/admin/provision</code> . Staff accounts require support area and academic department.
Assign routing	Admin updates support area and academic department using <code>PATCH /api/admin/users/:id/routing</code> .
Create FAQ	Admin creates question-answer entries that become searchable by the knowledge agent.
Upload document	Admin uploads text or a file. The backend chunks it and creates embeddings.
Review traces	Admin reads recent <code>OrchestratorTrace</code> records to understand assistant decisions.

Recent UI decision

The dashboard API key generator was removed from the admin interface. The backend model and endpoint still exist because the public API is still API-key protected.

12. Security And Permissions

The project has multiple layers of protection: password hashing, JWT authentication, role-based guards, service-level ownership checks, API-key protection for public endpoints, and throttling.

Security Area	Implementation
Password security	Passwords are hashed with Argon2id before storage.
JWT auth	Login returns a signed token containing user ID, school ID, role, support area, and academic department.
Route guards	<code>JwtAuthGuard</code> protects logged-in routes. <code>RolesGuard</code> restricts admin/staff/student endpoints.
Ticket access	Students see only their own tickets. Staff see only assigned-lane tickets. Admins see all.
Public API auth	<code>ApiKeyGuard</code> requires <code>x-api-key</code> and compares hashed keys in the database.
Rate limiting	Nest throttling is global, with specific limits on assistant and message routes.

13. Notifications, Email, And Uploads

Notifications

Notifications are created when tickets are created, claimed, updated, or receive messages. The dashboard reads unread counts from `/api/notifications/unread-count`. Ticket-related notifications can be marked read when the user opens the ticket.

Email

When a ticket is created, matching staff members with email addresses can receive email through the mail module. SMTP settings are configured through environment variables. If SMTP is not configured, the project can still run without email delivery.

File Uploads

Uploaded files are validated by MIME type and size, then stored on disk. The database stores file metadata so the app can list, open, and remove attachments while preserving ticket history.

14. Public API

The public API is designed for external integrations. It is protected by API keys and rate limiting. External systems can search knowledge and manage tickets through controlled endpoints.

Endpoints

- `GET /api/public/docs`
- `GET /api/public/knowledge/search`
- `POST /api/public/tickets`
- `GET /api/public/tickets/:ticketId`
- `PUT /api/public/tickets/:ticketId`
- `DELETE /api/public/tickets/:ticketId/attachments/:attachmentId`

Public API updates use an active admin account internally for ticket actions. This keeps the external API separate from normal browser JWT sessions.

15. Deployment And Running

Docker Compose is the recommended way to run the full stack. It starts PostgreSQL with pgvector, the NestJS API, the Next.js web app, and Nginx as a reverse proxy.

Docker Flow

```
docker compose up --build
  -> db starts PostgreSQL/pgvector
  -> api runs prisma db push
```

```
-> api runs seed script
-> api starts NestJS server
-> web starts Next.js app
-> reverse-proxy serves app on http://localhost
```

Important Environment Variables

Variable	Purpose
DATABASE_URL	PostgreSQL connection used by Prisma.
JWT_SECRET	Secret used to sign and verify JWT tokens.
GEMINI_API_KEY	Optional key for Gemini. Without it, local fallback logic is used.
NEXT_PUBLIC_API_URL	Frontend API base URL.
UPLOAD_DIR	Directory where uploaded attachments are stored.
BOOTSTRAP_ADMIN_*	Credentials and profile data for the seed-created admin account.

16. Testing And Project Health

The project includes unit tests and e2e tests for the API, plus lint and build checks for the web app.

Useful Commands

Command	Purpose
<code>cd web && npm.cmd run lint</code>	Runs frontend ESLint.
<code>cd web && npm.cmd run build</code>	Builds the production frontend.
<code>cd api && npm.cmd test -- --runInBand</code>	Runs backend unit tests.
<code>cd api && npm.cmd run build</code>	Builds the NestJS API.
<code>cd api && npm.cmd run test:e2e</code>	Runs e2e tests such as the health endpoint test.
<code>cd api && npx.cmd prisma validate</code>	Validates the Prisma schema.

Last health check result

Web lint/build passed. API tests/build/lint passed. Prisma validation passed. Dependency audits found vulnerabilities that should be reviewed before production deployment.

17. Recommended Improvements

1. Update vulnerable dependencies carefully and retest. Avoid blind force upgrades without checking breaking changes.
2. Update diagrams and README references that still mention dashboard API key generation if the UI will keep that feature hidden.
3. Add more knowledge documents for IT, registration deadlines, campus procedures, and common student problems.
4. Add more e2e tests for login, ticket creation, assistant escalation, staff claim, and admin routing.
5. Add API key listing, revocation, scopes, and audit logs before exposing key management in the UI again.
6. Add a password reset or admin password rotation workflow for production use.
7. Add stronger file scanning or storage isolation if attachments will be used in a real deployment.

18. Summary

This project is a complete support portal with a real backend, real database design, role-based permissions, ticket management, knowledge ingestion, and a multi-agent assistant. The most

important design choice is that the assistant is grounded in trusted knowledge and escalates uncertain cases instead of guessing.

In practical use, students get answers or tickets, staff get scoped queues, admins manage operations, and the system records traces so the team can understand how the assistant is behaving. That makes the project more than a CRUD app: it is an operational workflow system with AI support.